

POC Documentation — AI Career LaunchPad

File: [poc_documentation.pdf](#)

Project: AI Career LaunchPad

Builder: Marco Martins

Context: Ironhack AI Consulting & Integration Bootcamp — Final Capstone Project

Status:

WF1 (Career Compass) — Working end to end.

WF3 (Application Engine) — Working end to end.

TABLE OF CONTENTS

POC Documentation — AI Career LaunchPad	1
1. What the POC Does	3
2. Tools Used and Why	3
3. WF1 — Step by Step	4
3.1 Input	4
3.2 Binary Handling	4
3.3 Parallel LLM Extractions	5
3.4 Portrait Assembly	5
3.5 Profile Matching	5
3.6 Report Generation	6
3.7 Output	7
4. WF3 — Step by Step	7
4.1 Input	7
4.2 Binary Handling and Parsing	7
4.3 JD Extraction	7
4.4 Job Fit Analysis	8
4.5 Application Generation	8
4.6 Output	8
5. AI Capability Demonstrated	9
6. Known Limitations (POC vs Production)	9
7. How to Reproduce / Run the POC	11
Prerequisites	11

WF1 — Career Compass

11

WF3 — Application Engine

11

Notes

12

1. What the POC Does

AI Career LaunchPad is a two-workflow AI system that helps professionals transitioning into AI consulting understand how to position their prior experience as a strategic asset.

Workflow 1 — Career Compass (WF1) Takes three PDF inputs: a LinkedIn profile, a CliftonStrengths report, and a completed Self-Reflection Workbook. Runs three parallel LLM extractions (career depth, strengths, working style and transition motivation), assembles a structured candidate portrait, scores the candidate against 8 AI consulting career archetypes, validates the scoring logic programmatically, and generates a five-tab HTML Career Compass report. The report includes a positioning statement, elevator pitch, profile matches, SWOT analysis, energy vs performance mapping, concern responses, transition strategy scenarios, and a personalised 90-day next steps plan with certifications and learning recommendations.

Workflow 3 — Application Engine (WF3) Takes two file inputs: a Career Compass JSON export (output from WF1) and a Job Description PDF. Extracts structured data from both, runs a multi-dimension job fit analysis, generates a tailored cover letter and seven interview questions with Clifton-grounded answer guidance, and outputs a three-tab HTML Application Engine report.

2. Tools Used and Why

Component	Tool	Reason
Workflow automation	n8n Cloud (self-hosted v2.13.2)	Visual workflow builder with native code nodes, webhook triggers, and multi-item processing. No vendor lock-in on the AI layer.
LLM	OpenAI gpt-4o (WF1) / gpt-4o-mini (WF3)	gpt-4o used for complex extraction and report generation tasks requiring high fidelity. gpt-4o-mini used for JD extraction and application outputs where cost efficiency is the priority.
PDF extraction	n8n Extract From File node	Native n8n node. No external dependency. Handles multi-item binary pipelines.

Component	Tool	Reason
Knowledge base	Hardcoded string in Code node	POC decision. Eliminates vector database dependency and latency. 8 profiles fit comfortably in the LLM context window.
Output delivery	n8n Respond to Webhook (text/html)	Single-step delivery. No frontend framework required. Output is a self-contained HTML page rendered directly in the browser.
Credentials	n8n credentials store	OpenAI API key stored securely. Not hardcoded in any node.

3. WF1 — Step by Step

3.1 Input

Form Trigger accepts three file uploads:

- **LinkedIn_PDF** — candidate's LinkedIn profile exported as PDF
- **Clifton_PDF** — full CliftonStrengths report PDF (34-profile version with "Why Your X Is Unique" sections)
- **SelfReflection_PDF** — completed Self-Reflection Workbook PDF (6 blocks, ~37 questions)

3.2 Binary Handling

Split Binary (Code node): Splits the three uploaded binaries into three separate items, each carrying its binary field named **file**. Source label attached to each item's JSON (**linkedin**, **clifton**, **selfreflection**).

Extract PDF: A single Extract From File node processes all three items automatically due to n8n's item-based processing model. Outputs **text** field on each item.

Combine Candidate Inputs (Code node): Reassembles extracted text by position (items[0] = LinkedIn, items[1] = Clifton, items[2] = Self-Reflection). Cleans the Clifton text by stripping boilerplate tail content at the "What did you read that surprises you?" marker. Parses the

Self-Reflection PDF into 6 named blocks using marker-based extraction. Extracts structured search parameters from Block 6 (timeline, employment preference, relocation, working mode). Outputs all fields as a single item. Runs validation and surfaces warnings for missing content.

3.3 Parallel LLM Extractions

Three LLM nodes run in parallel, each receiving the combined candidate inputs:

Career Depth Extraction (gpt-4o, temp 0.2): Extracts career arc summary, domain expertise, transferable skills, AI narrative anchors, signature achievements, differentiators, and subject matter preferences. System prompt instructs strict factual extraction — LinkedIn for evidence, self-reflection Block 1 for intent. JSON output mode ON.

Strengths Extraction (gpt-4o, temp 0.2): Extracts the top 5 Clifton Strengths with personalised insights (from "Why Your X Is Unique" sections, not generic definitions), deploy-now statements, learning accelerators, and development risks. Extracts strengths cluster analysis, career anchors, working style, energy vs performance split, and hard avoids. System prompt enforces use of personalised insight sections and second-person voice. JSON output mode ON.

Working Style and Transition (gpt-4o, temp 0.3): Extracts bootcamp insights, transition motivation (pull vs push), identity preference, unspoken concern, full concerns array with Clifton-grounded bridge actions, niche hypothesis, daily work preference, stepping stone signal, lifestyle constraints, and search parameters. System prompt enforces minimum 3 concerns with specific Clifton strength mappings and 3 bridge actions each. JSON output mode ON.

3.4 Portrait Assembly

Merge — Portrait Inputs: Waits for all three LLM extractions to complete. 3-input merge node.

Assemble Candidate Portrait (Code node): Reads LLM outputs from `items[x].json.message.content` (JSON output mode produces a JavaScript object at this path). Assembles { `career_depth`, `strengths`, `transition` } portrait object. Validates that key fields are present. Outputs `candidate_portrait` as a JavaScript object (not a string).

3.5 Profile Matching

Load KB Profiles (Code node): Reads the candidate portrait and attaches the full 8-profile knowledge base as a hardcoded string. Outputs `candidate_portrait` (JSON.stringify'd) + `kb_profiles`.

KB Profile Matching (gpt-4o, temp 0.2): Scores all 8 profiles across 4 dimensions (`skills_match`, `strengths_alignment`, `preference_fit`, `anchor_alignment`), each 0–25. Hard constraint rules cap profiles that violate the candidate's hard avoids (single violation: cap

at 55; double violation: cap at 35). Returns `all_scores` array and `top_matches` array (top 3 by final capped total). JSON output mode ON.

Validate Profile Matching (Code node): Programmatically re-applies hard constraint caps to all 8 scores regardless of what the LLM returned (LLM flags violations but does not always apply the arithmetic cap). Re-sorts all scores by `final_total` descending. Rebuilds `top_matches` array to guarantee the 3 highest final totals appear in rank order. Records whether ranking was corrected.

3.6 Report Generation

Prepare Report Inputs (Code node): Retrieves portrait from Assemble Candidate Portrait node and matches from Validate Profile Matching. Serialises both to strings via `JSON.stringify`. Outputs `candidate_portrait` + `kb_matches`.

Report Sections (gpt-4o, temp 0.5): Generates core report content — `who_you_are`, `career_reframe_narrative`, `what_makes_you_rare`, `top_profiles` with gaps, SWOT, `gap_analysis`, `energy_vs_performance`. All claims must reference specific portrait fields. JSON output mode ON.

Prepare Positioning Inputs (Code node): Retrieves `report_sections` output and passes through portrait and `kb_matches` for downstream nodes.

Positioning and Strategy Part 1 (gpt-4o, temp 0.6): Generates `positioning_statement` (headline + supporting narrative), `elevator_pitch` (who/do_what/so_they_can/full_pitch), `unspoken_concern_response`, and `presentation_development_note`. JSON output mode ON.

Prepare Part 2A Inputs (Code node): Assembles Part 1 output with portrait, `kb_matches`, and `report_sections` for the transition strategy node.

Positioning and Strategy Part 2A (gpt-4o, temp 0.6): Generates three transition strategy scenario cards (`need_now`, `have_runway`, `part_time`), each with scenario-specific free actions formatted as `[action] – powered by [Strength] – addresses [SWOT element]`. Determines `recommended_scenario` from `stepping_stone_signal` and timeline. JSON output mode ON.

Prepare Part 2B Inputs (Code node): Assembles Part 1 and Part 2A outputs with portrait and `kb_matches` for the next steps node.

Positioning and Strategy Part 2B (gpt-4o, temp 0.6): Generates the full `next_steps` section — `day_1` through `day_90` milestones, certifications (Ironhack priority certifications + domain-specific additions), and selective reading plan (books, articles, self-paced, build-your-own-lab). JSON output mode ON.

Assemble Positioning JSON (Code node): Merges Part 1, Part 2A, and Part 2B outputs into a single assembled positioning object. Validates all expected keys are present. Wraps in `{ message: { content: assembled } }` for compatibility with downstream node.

Assemble Report JSON (Code node): Merges Report Sections output with Assemble Positioning JSON output. Positioning keys win on collision. Validates final report structure. Outputs `{ report }`.

3.7 Output

Build HTML Output (Code node): Reads the assembled report JSON. Builds a five-tab HTML page using JavaScript string operations — no markdown parsing, no template engine. Tabs: Your Compass / Your Best Profile / Your Reality Check / Your Move / Next Steps. Includes a textarea in Tab 5 (Next Steps) containing the Career Compass JSON payload for copying into WF3.

Output (Respond to Webhook): Returns the HTML string as `text/html; charset=utf-8`.

4. WF3 — Step by Step

4.1 Input

Form Trigger accepts two file uploads:

- **CareerCompass** — the Career Compass JSON file exported from WF1 Tab 5
- **JobDescription** — the job description as a PDF

4.2 Binary Handling and Parsing

Split Binary (Code node): Validates that both binary fields are present. Passes them downstream as a single item with both binaries intact.

Parse Career Compass (Code node): Reads the **CareerCompass** binary using `helpers.getBinaryDataBuffer()` (required for n8n filesystem binary mode). Parses the JSON string. Attaches **career_compass** to the item JSON. Passes binary through so Extract JD Text can still access **JobDescription**.

Extract JD Text: Extracts text from the **JobDescription** binary PDF.

Combine WF3 Inputs (Code node): Combines JD text with **candidate_portrait** and **profile_matches** from the Career Compass JSON. Validates all three fields are present.

4.3 JD Extraction

LLM — JD Extraction (gpt-4o-mini, temp 0.2): Extracts structured fields from the raw JD text: role title, company, location, remote/hybrid/onsite, contract type, seniority level, salary range, industry, company size/type, responsibilities, must-have requirements,

nice-to-have requirements, application deadline, keywords, and extraction notes. Returns null for any field not explicitly stated in the JD.

4.4 Job Fit Analysis

Prepare Fit Inputs (Code node): Combines JD structured data with candidate portrait and profile matches.

LLM — Job Fit Analysis (gpt-4o-mini, temp 0.2): Scores the candidate against the JD across four dimensions (skills_fit, experience_fit, culture_style_fit, seniority_fit, each 0-25). Identifies the closest WF1 profile match and mapping confidence. Generates strong_fit_angle, gap_analysis with how-to-talk-about-it guidance, Clifton deployment recommendations per strength, mismatch flags, and overall recommendation with rationale.

4.5 Application Generation

Prepare Application Inputs (Code node): Passes fit analysis, JD structured data, and candidate portrait to cover letter and interview question nodes.

LLM — Cover Letter (gpt-4o-mini, temp 0.6): Writes a four-paragraph cover letter in first person, maximum 350 words. Paragraphs: hook (references strong_fit_angle), evidence (names specific achievements), transition narrative (references identity_preference.framing_in_their_words), close (clear ask, confident tone). Returns full_text and word_count.

LLM — Interview Questions (gpt-4o-mini, temp 0.4): Generates 4 behavioural and 3 technical/role-specific questions grounded in the actual JD. Each question includes: why_they_ask_it, answer_angle, experience_to_lead_with, clifton_strength_to_deploy, and gap_mitigation where relevant.

4.6 Output

Merge: Combines Cover Letter and Interview Questions outputs into two items.

Inject Prepare Data (Code node): Re-attaches fit_analysis, jd_structured, and candidate_portrait to both merged items so Build HTML Output can access all data from a single input.

Build HTML Output (Code node): Builds a three-tab HTML page. Tabs: Job Fit Report / Cover Letter / Interview Prep. Job Fit tab includes score banner, profile mapping, strong fit angle, dimension cards, gap cards with severity ratings, Clifton deployment cards, and mismatch flags. Cover Letter tab includes formatted letter with paragraph structure and a copy-to-clipboard textarea. Interview Prep tab renders seven question cards with answer guidance.

Output (Respond to Webhook): Returns the HTML string as `text/html; charset=utf-8`.

5. AI Capability Demonstrated

The POC demonstrates four distinct AI capabilities working in sequence:

Structured extraction from unstructured documents. Three separate LLM nodes extract specific, schema-constrained JSON from free-form PDF text (LinkedIn profiles, CliftonStrengths reports, self-reflection workbooks). The prompts enforce factual extraction discipline — the LLM does not invent or infer beyond what is in the source documents.

Multi-dimensional scoring and matching. The KB Profile Matching node scores a candidate portrait against 8 predefined archetypes across 4 dimensions simultaneously. Hard constraint rules applied at both the LLM prompt level and in a downstream validation Code node ensure scoring arithmetic is correct regardless of LLM output drift.

Personalised long-form generation. Five sequential LLM nodes generate the Career Compass report sections. Each node receives the full candidate portrait and profile matches as context, and is prompted to generate content that could only apply to this specific candidate. The prompts include banned-word lists, minimum cardinality rules on arrays, and explicit grounding requirements (every claim must reference a specific portrait field by name).

Context-aware application generation. WF3 takes the Career Compass output (already rich with structured candidate data) and combines it with a specific JD to generate a fit score, cover letter, and interview preparation that references the intersection of both. The cover letter hook is explicitly derived from the `strong_fit_angle` produced by the job fit analysis.

6. Known Limitations (POC vs Production)

Limitation	Impact	Production Resolution
Knowledge base hardcoded as string in Code node	Cannot update profiles without editing workflow. No semantic search.	Replace with Pinecone or Supabase vector store. Profiles become embeddable documents.
No user authentication or	Anyone with the form URL can submit. No user identity.	Add auth layer (n8n webhook auth, or dedicated frontend with Clerk/Auth0).

Limitation	Impact	Production Resolution
session management		
No data persistence	Career Compass output is a one-time browser page. If closed, the data is gone.	Add database node (Supabase, Airtable) to store report outputs against user ID.
Career Compass JSON passed as manual file upload into WF3	Friction in the WF1 → WF3 handoff. User must copy JSON from textarea and upload as a file.	Connect WF1 output directly to WF3 via shared session ID or database lookup. Eliminate the file step.
No error handling on LLM node failures	If any LLM node returns malformed JSON, the workflow fails silently or throws an uncaught error.	Add try/catch in each Code node that processes LLM output. Add fallback branch with error response.
PDF extraction quality depends on PDF structure	LinkedIn PDFs from different export methods produce different text quality. Some CVs export as image-based PDFs and produce empty text.	Add pre-processing step to detect empty extraction and return a user-facing error. Document PDF export instructions for users.
No rate limiting	High volume of simultaneous submissions would hit OpenAI rate limits.	Add queue node or rate limiter in n8n. Consider gpt-4o-mini for early extraction steps to reduce cost and latency.
WF2 (Job Search Engine) not built	No automated job discovery. Candidates manually source JDs to run through WF3.	Build WF2 using Adzuna or SerpAPI, triggered by Career Compass output.

Limitation	Impact	Production Resolution
gpt-4o used throughout WF1	Higher cost per run (~\$0.15–0.30 per submission at current token volumes).	Benchmark gpt-4o-mini on extraction nodes. Reserve gpt-4o for report generation nodes where quality matters most.

7. How to Reproduce / Run the POC

Prerequisites

- n8n Cloud account or self-hosted n8n instance (v2.13.2 or later recommended)
- OpenAI API key with access to [gpt-4o](#) and [gpt-4o-mini](#)
- n8n credential named [OpenAi account](#) configured with the API key

WF1 — Career Compass

1. Import [WF1 - Career Compass v2.json](#) into n8n.
2. Activate the workflow.
3. Open the Form Trigger URL (displayed in the Form Trigger node).
4. Upload:
 - LinkedIn profile exported as PDF (File → Save as PDF from the LinkedIn profile page)
 - Full CliftonStrengths report PDF (34-page version — must include "Why Your X Is Unique" sections for each strength)
 - Completed Self-Reflection Workbook PDF (6 blocks, printed and filled, then scanned to PDF — or completed digitally in a fillable PDF)
5. Submit the form.
6. Wait 60–120 seconds for the pipeline to complete (7 LLM calls in sequence and in parallel).
7. The browser will display the Career Compass HTML report.
8. Navigate to Tab 5 (Next Steps), scroll to the bottom, click the textarea, select all, and copy the Career Compass JSON payload.
9. Save the copied JSON as a [.json](#) file for use in WF3.

WF3 — Application Engine

1. Import [WF3 - Job Application Engine V1.json](#) into n8n.
2. Activate the workflow.

3. Open the Form Trigger URL.
4. Upload:
 - Career Compass JSON file (.json file saved from WF1 Tab 5)
 - Job Description PDF (copy the JD text into a Word doc and export as PDF, or print-to-PDF from the job board page)
5. Submit the form.
6. Wait 30–60 seconds.
7. The browser will display the Application Engine HTML report with three tabs: Job Fit Report, Cover Letter, Interview Prep.

Notes

- The Self-Reflection Workbook is a separate PDF document containing 6 blocks of structured questions about career depth, working style, bootcamp experience, transition motivation, career integration, and search parameters.
- For best results, the LinkedIn PDF should be the full profile export, not a printed version of the public profile page.
- The CliftonStrengths PDF must be the full report with personalised insight sections. The top-5 summary card (one page) is not sufficient.
- Demo recording and walkthrough video will be added to this document once the presentation is finalised.

*AI Career LaunchPad — Ironhack AI Consulting & Integration Bootcamp — Week 9
Capstone*